

Apuntes SSOO

Víctor Manuel Peiró Martínez

FIIS 2ºA

Sistemas Operativos

Tema 4 - Gestión de Memoria Pt2

Memoria Real

La memoria real o física del ordenador es la RAM.

Los SSOO de propósito general trabajan sobre todo con memoria virtual pero hay situaciones donde es necesario trabajar con procesos que estén cargados completamente en memoria.

Sistemas de Soporte Vital: Sistemas donde es necesario trabajar con procesos que estén cargados en memoria

Swapping: El SSOO concede toda la memoria física a el proceso en ejecución y copia a disco los datos del proceso expulsado.

MMU: Mecanismo HW de protección que evita que un proceso acceda a la dirección de otro proceso.

- Esa comprobación hay que hacerla en operación de lectura o escritura
- Consiste en tener un registro base con la dirección inicial del proceso y otro límite.
- En cada acceso a RAM, la MMU comprueba que la dirección no se sale de ese rango. De lo contrario, se produce una interrupción

El kernel tiene una tabla con todos los pares base/límite y actualiza los registros de la MMU en cada cambio de contexto

Técnicas de asignación de Memoria

En los sistemas con memoria real, hay que proceder al reparto entre todos los procesos cargados cuyos tamaños pueden ser muy diferentes.

Es muy improbable que haya un hueco de tamaño exacto cuando se carga un proceso en memoria.

Fragmentación: Se generan pequeños huecos no ocupados pero que no se pueden utilizar.

2 técnicas. Bitmap y Listas Encadenadas

Bitmap: Se divide la memoria en pequeñas zonas y se marca en un vector de bits si están ocupadas o no.

- Se necesitan 1 bit por cada 32 bits. Un sistema de 4GB tiene un bitmap de 128MB
- Bitmap tiene un problema de rendimiento
- Cuando el SSOO carga un nuevo proceso, tiene que recorrer el bitmap desde el principio hasta encontrar el hueco

Listas Doblemente Encadenadas: Guarda los datos de los fragmentos de memoria ocupados por procesos y los que están libres.

- Ese mucho más recorrido recorrer la lista de este tipo que el bitmap y además consume mucho más espacio en memoria
- El inconveniente es que es memoria dinámica que hay que gestionar desde el SSOO y puede ser complejo para equipos empujados de bajas prestaciones.

Algoritmos de Asignación de Memoria Real

First Fit: Asigna el proceso en el primer hueco en el que cabe

Best Fit: Se busca el hueco más pequeño en el que cabe el proceso

Next Fit: Igual que First Fit pero se empieza a recorrer desde el último espacio asignado

Worst Fit: Se busca el hueco más grande. Inconveniente: Hay que recorrer toda la lista

Quick Fit: Se mantienen varios juegos estandarizados. Al terminar el proceso, se hace un merge con el espacio de memoria libre más próximo.

Segmentación

Segmentación: Técnica de reubicación que emplea la idea de puntero a una base reubicable

Permite usar distintos segmentos de memoria para código, los datos o el stack.

En x86 hay 4 segmentos. 3 definidos y uno de uso libre.

Añade una capa de protección adicional porque el programa no puede escribir en la zona de código o ejecutar en la zona de datos.

Cada proceso tiene su propia tabla de segmentos en una zona de memoria del kernel.

Memoria Virtual

Solo trabajan con memoria real los sistemas empotrados muy pequeños(Arduino) o aquellos de tiempo real que por requisitos de rendimiento necesitan todos los proceso cargados.

En equipos con memoria virtual, la RAM respecto al disco como la caché a la RAM.

Para gestionar esto:

- El SO tiene que conseguir gestion transparente al proceso y no condicione la programación
- La traducción de direcciones virtuales a direcciones físicas tiene que ser muy rápida. Por ellos los procesadores disponen de **circuitos MMU(Memory Management Unit)**
- Los tiempos de acceso a disco magnético son unos 6 órdenes de magnitud superiores a los de la RAM y los fallos tiene que reducirse al mínimo inevitable

Todos los sistemas utilizan técnicas de paginación.

Paginación: La memoria virtual, la memoria física y los accesos a y desde disco para cargar y escribir páginas se hacen usando páginas de tamaño fijo.

Los procesos ven un mapa de memoria virtual completo que se divide en páginas.

Cada página se instancia en un marco de memoria física

El SO se encarga de la gestión y preparar los datos para realizar traducciones

Cuando una pagina no esta cargada en RAM se dice que se ha producido un fallo de pagina

- Se resuelve buscando un marco libre o expulsando uno ocupado en el que cargar dicha página

Elle tiempo de tratamiento del fallo, el micro puede ordenar del orden de 50 millones de instrucciones por lo que el proceso se bloquea para dejar la CPU a otro

La memoria virtual funciona como nivel de indirección

Conjunto Residente: Páginas de un proceso cargadas en RAM

Swap: Partición especial para el mapeo de direcciones

La memoria virtual con paginación añade flexibilidad al poder asignar espacio físico no contiguo en la RAM. Se reduce la fragmentación al mínimo

Para que la memoria virtual pueda funcionar, es necesario que la correspondencia entre página virtual y marco físico se almacenen en alguna estructura.

Hay que tener una tabla de correspondencia por cada proceso instanciado y hay que actualizarla cada vez que hay una carga o expulsión

El kernel mantiene una página por proceso, dentro de su propio espacio de memoria.

Esta tabla forma parte de los metadatos de gestión de cada proceso

Tablas de un Nivel

Tiene que estar completamente cargada en memoria cuando el proceso está cargado

Cada proceso tiene su tabla de páginas, que reside en el espacio de trabajo del kernel

Por cada entrada de la tabla de páginas de un proceso, además del número de marco hay otros bits de control:

- **Referenced:** Página que ha sido referenciada recientemente y se utilizan para el algoritmo de sustitución de páginas
- **Modified:** Indica los contenidos que se ha modificado y que deberá ser escrita antes de ser expulsada de disco
- **Protection:** Indica si la página es de código, se puede escribir
- **Present/Absent:** Indica si la entrada es válida.
- **Caching Disabled:** Indica si para acceder a esa página hay que inhabilitar la caché

Tablas Multinivel

Añaden un nivel de indirección

Se crean entradas de segundo nivel para las entradas de primer nivel que ya estén ocupadas

TLB (Translation Lookaside Buffer)

Imaginemos una tabla de 5 niveles:

- Por cada acceso a memoria del proceso tendría que realizar 5 accesos solo para saber si la tabla está cargada
- Esto no puede realizarse por SW por ello el MMU dispone de una memoria asociativa especial llamada la TLB para llevarlo a cabo.

La TLB es pequeña y muy rápida y mantiene la relación entre números de página y marcos accedidos más recientemente.

Cada vez que la CPU escribe una dirección virtual, se presenta a la TLB.

Es memoria asociativa

Si está presente la dirección, realiza la traducción de inmediato, sino se produce fallo en la TLB.

- Se produce una interrupción que despierta al kernel.
- Si la página está en RAM, el kernel toma esos datos, actualiza la TLB y puede realizarse la traducción

La TLB se carga cada vez que hay un cambio de contexto

Un puntero llamada **PTBR(Page Table Base Register)** apunta a la dirección de la tabla de proceso actual

- La TLB es parte la MMU. Incluye gestión de segmentación, la actualización de los bits de estado de la tabla de direcciones, la composición de direcciones físicas y la generación de interrupción
- La TLB es muy rápida si hay acierto
- La TLB típica tiene 64 entradas
- La localidad permite que los fallos en la TLB sean muy bajos
- Cada vez que hay un cambio de contexto, se actualiza la TLB
- Durante el arranque del sistema, la función del TLB está anulada porque no existe tabla de memoria

TLB con Cache

La TLB sólo traduce direcciones virtuales a físicas. La caché solo trabaja con direcciones físicas.

El SO puede programar la TLB, el caché es inaccesible.

La TLB se sitúa a la salida de la CPU, la caché entre la RAM y la TLB

El fallo de cache no es bloqueante

Fallo de Pagina

Fallo De Página: Se produce cuando la página a la que quiere acceder la CPU corresponde con una página no cargada en memoria.

Cuando esto ocurre, el gestor de memoria virtual busca un marco libre y se lo asigna a la página en cuestión y lanza una operación de DMA.

Como es una operación muy costosa, el proceso se bloquea hasta que termine la carga y se cede a la CPU.

También puede ocurrir que esté toda la memoria ocupada. El gestor tiene que decidir qué página expulsar para dejar un marco libre.

Tratamiento Completo de Fallo de Pagina:

1. MMU genera una interrupción que despierta al kernel. Este salva los registros del proceso en la pila e invoca un gestor de memoria
2. El SO busca en la tabla de gestión de memoria física si hay un marco libre. Si lo hay, lo asigna a la página, lanza el proceso y lo bloquea
3. Si no lo hay se aplica un algoritmo de reemplazo. Si la página está sucia, tiene que escribirla primero en la zona de swap y cargar la nueva
4. Cuando termine la carga, la instrucción que produjo el fallo se restaura y el proceso ya puede acceder a dato que necesitaba
5. Cuando el planificador lo decida, el proceso se restaura y sigue con la ejecución

Algoritmos de Reemplazo de Pagina

NRU: Se expulsa la página que lleva más tiempo sin ser usada.

- Se usan los bits M(modificada) y R(Referenciada)
- Las páginas con configuración MR = 00 son las primeras en expulsarse. Luego las 01 y las 11.
- Se borran con una interrupción tick

FIFO: Se elimina la página que lleva más tiempo en memoria

- Usa una lista unidireccional en la que se borra la página de un extremo y se añade a el otro

Segunda Oportunidad: Una página referenciada gana una vida extra

- Si la página tiene el bit R a 1, se pone a 0 y se coloca en el extremo opuesto de la lista de entrada

Gestión del Área de Swap

Zona de Swap: Partición especial del disco para grabar páginas expulsadas o cargar páginas de un nuevo proceso

Como todas las páginas tienen el mismo tamaño, tiene una estructura plana.

Cuando el ordenador arranca esta zona está vacía.

Cuando se crea un proceso, se asigna una porción de la swap a sus páginas.

Se guarda una zona del swap en el BCP

Antes de que cargue el proceso es posible crear su imagen de memoria en la zona de swap e ir cargando desde ella.

La otra es solo cargar solo en memoria e ir escribiendo el en swap cuando sea necesario

Se distinguen varias zonas en el área de swap para texto, datos y pila.

Ejercicios

Pasos:

- Tamaño del Marco = Tamaño de Página = Tamaño Offset (en bytes)
- Tam Virtual Address = 2^n bytes. n = bits sistema
- Tam Physical Address = 2^r bytes
- # paginas = N° bits VA - n° bits offset
- # marcos = N° bits PA - n° bits offset

Ej 1: Rellenar la siguiente tabla para indicar el tamaño en bits de cada ítem

- Sistema de 32 bits, pag de 4 kB, RAM de 1GB**
- Sistema de 32 bits, pag de 16 kB, RAM de 2GB**
- Sistema de 64 bits, pag de 16 kB, RAM de 16GB**

Apartado	VA	PA	# paginas	# marcos	offset
a)	32	30	32-12 = 20	30 - 12 = 18	12
b)	32	31	18	17	14
c)	64	34	50	20	14

Ej 2: Un sistema tiene direcciones virtuales de 16 bits y direcciones de memoria fisica de 24 bits. Usa una tabla virtual de 2 niveles, 4 bits para L1 y 4 bits para L2. Cual va a se el offset.

Como espacio de direcciones de 16 bits:

L1	L2	Offset
4	4	16 - 4 - 4 = 8

8 bits para el offset

Cuántas paginas tiene la memoria virtual

$$\frac{\text{Espacio Total}}{\text{Tam Pagina}} = \frac{2^{16}}{2^8} = 2^8 \text{ paginas}$$

Tam Memoria Fisica = 2^{24} bytes = 16MB

Ej 3: En un ordenador de los años 60, tenemos direcciones virtuales de 16 bits, paginas de 1kB, 5 bits de numero de marco. Tam Max de memoria fisica

Bits Offset = 1 kB = 2^{10} bytes → 10 bits para offset

Bits Mem Fisica - bits offset = bits num marcos

Bits Mem fisica = 5 + 10 = 15 bits

Ej 4: En un sistema de 32 bits con memoria virtual y paginas de 64kB la TLB tiene las siguientes entradas:

Pagina Virtual	Marco	Valida
13F1H	1876H	1
0018H	021FH	1
1E10H	0012H	1
3110H	0891H	0
0321H	0002H	0

0005H	1091H	1
0023H	0C01H	1

Traducir las siguientes direcciones virtuales en físicas

VA: 13F1 78BF H

PA: 1876 78BF H

VA: 0023 5574 H

PA: 0C01 5574 H

VA: 1002 9931 H

PA: Pagina no cargada

VA: 3110 9F00 H

PA: Pagina no Valida

VA: 1E10 9977 H

PA: 0012 9977 H

Tam Pag = 64kB = 2^{16} bits → 16 bits para el offset. Los últimos 16 bits no cambian

Gestión de Imagen de Memoria

Imagen de Memoria: Conjunto de información necesaria para ejecución.

Se crea a partir del ejecutable y este es producto de la cadena de compilación y linkado.

Contenido de Ficheros Objeto:

- Cabecera que describe cómo se organiza
- Texto: Código Máquina y constantes del módulo
- Variables Globales
- Información de reubicación
- Tabla de simbolos externos
- Información de depuración

Linux maneja esto con una estructura mm_struct

Para construir el ejecutable, el linker necesita conocer qué librerías y módulos contienen referencias externas. Estos módulos se reubican al final del ejecutable.

Si el montaje es estático, todo el código de las librerías se incluye en el ejecutable, aunque no se usen las funciones.

El montaje dinámico solo incluye una pequeña rutina capaz de localizar a la librería dinámica e invocar.

La librería se carga la primera vez que un proceso la requiere

El kernel esta permanente cargado en la RAM.

En todos los procesos, se mapea en la tabla de memoria individual.

Copy On Write(COW): Si tenemos 2 instancias del mismo ejecutable, las paginas de memoria virtual se mapean en los mismos marcos.

Si durante la ejecución se alteran los datos, la pagina afectada se copia en un marco nuevo.

Tema 5 - Gestión de Ficheros

Introducción

Cintas Magnéticas: Almacenamiento no Volátil para guardar datos y programas. Funcionan con tambores giratorios.

El rey del almacenamiento secundario son los **discos magnéticos** ya que su cinta de lectura/escritura no tenía que ser secuencial y tenía mas velocidad

- Primer Modelo: IBM 350

Los **Discos de Estado Sólido** han reemplazado a los discos magnéticos y se utilizan otros tipos de memoria como las tarjetas SD

Sin embargo, los programas de usuarios son inmunes a los cambios de tecnología porque usan ficheros.

En Unix, casi todo se puede tratar como fichero

Fichero

Fichero: Abstracción de Almacenamiento Secundario, una secuencia de bytes que no afecta al SO.

La información se presenta como una serie de registros consecutivos del mismo tamaño con un puntero que permite acceder de forma aleatoria a cualquiera de ellos.

Un Fichero tiene que cumplir 3 condiciones:

1. Almacenar una cantidad ilimitado de información
2. Sobrevivir al proceso que lo creo
3. Permitir el acceso simultáneo de varios procesos

Bloque: Unidad mínima de transferencia entre el disco y el ordenador

- En sistemas de memoria virtual coincide con el tamaño de la página. En Windows y Linux son 4kB que son 8 sectores.

La traducción de número de bloque a dirección física la hace el driver.

Agrupación: Conjunto de Bloques.

- En Unix. Agrupación = Bloque

La estructura de un fichero consiste en el mapa de agrupaciones que ocupa.

Mapa del Fichero: Mapa de Agrupaciones

El hecho de que el fichero ocupe agrupaciones enteras produce un problema de fragmentación interna(espacio desaprovechado).

El problema de la organización por bloques es que cada transferencia requiere 4096 bytes(4 kB) aunque solo se modifique un bloque.

Metadatos de Un Fichero

Un fichero no solo consiste en la información que almacena sino que contiene metadatos.

Metadatos: Datos imprescindibles sobre un fichero

Identificador: Clave única que identifica el fichero

Nombre: Cadena de Caracteres que debe de ser única dentro de un directorio.

- En Unix, la extensión es decorativa

Fechas: De Creación, último acceso y modificación

Tipo: Regular, de E/S, Directorio

Mapa del Fichero: Conjunto de Agrupaciones que ocupa

Propietario: En Unix se identifica por el UserID o GroupID

Protección: Permisos del fichero

Tamaño: Bytes de información útil del fichero

Comandos de Visualización:

- En Linux: ls
- En MS-DOS: dir

Los metadatos no se guardan en el propio fichero sino en estructuras especiales:

- Directorios
- Descriptores Fisicos de Ficheros (DFF)

Master File Table: DFF de Windows

i-nodos: DFF de Unix

- Se identifica por un número único que tambien es el identificador del fichero
- La relación i-nodo/fichero es 1 a 1.

Disco Magnético

Superficie giratoria sobre la que se ha depositado un material ferromagnético para lectura y escritura con un electroimán. Se distinguen el 0 del 1 según el estado de imantación

Sector: Unidad Mínima de Lectura/Escritura (Típicamente 512 bytes)

- La cabeza no puede leer un byte en particular sino un sector entero

Pista: Conjunto de Sectores

- Todas la pista tienen el mismo diámetro
- Se van añadiendo más discos. Todas las pistas de distintas superficies construyen un cilindro

Para leer/escribir en un sector, la cabeza se desplaza radialmente hasta situarse sobre la pista y espera a que pase el sector deseado

Un preámbulo en cada sector permite identificar a la cabeza el número del sector

Directorio

Directorio: Clase de fichero que guarda información sobre otros ficheros

- Se encuentra la relación entre el nombre de un fichero y su DFF.

Los subdirectorios permiten agrupar ficheros de características similares.

Los directorios siguen una estructura jerárquica que apuntan a otros subdirectorios o ficheros normales.

Cada fichero recibe un nombre local que debe de ser único dentro de su carpeta

La función del directorio es mantener el árbol de descriptores. Cada entrada del directorio solo tiene 2 datos:

- Nombre del Fichero
- DFF

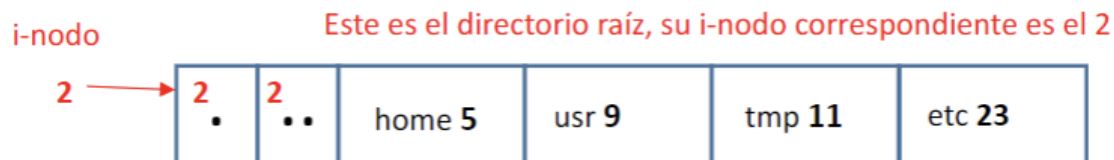
El i-nodo del directorio raíz siempre es 2 y es padre de sí mismo

Cada directorio tiene dos entradas especiales:

- . : Apunta a si mismo
- .. : Apunta a el directorio padre

Ejemplo: Imaginemos que queremos encontrar el archivo hola.txt que se encuentra en /home/pepe/literatura

Comenzamos en directorio raíz:



Como queremos ir al directorio home, el i-nodo cambia a 5 ya que es el directorio al que apunta y para el resto.

porque home
apunta a 5
5

apunta al padre

5 .	2 ..	pepe 31	pili 26	luisa 39	juan 24
--------	---------	---------	---------	----------	---------

apunta a sí mismo y es hijo de 5

31

.31	5 ..	correo	UTAD 105	literatura 7
-----	---------	--------	----------	--------------

literatura apunta 7, 7 es hijo de 31

7

7 .	31 ..	MioCid 51	hola.txt 12
--------	----------	-----------	-------------

Enlaces

En Unix, existe la posibilidad de crear enlaces para un mismo fichero

Enlace Simbólico: Crea una nueva entrada en el directorio hacia el fichero apuntado
Si el enlace se borra, el fichero original no es afectado.

Sistemas de Ficheros

Sistema de Ficheros: Estructura de Información que permite manejar los ficheros de una unidad de almacenamiento.

Partición: Sistema de Fichero de Unix

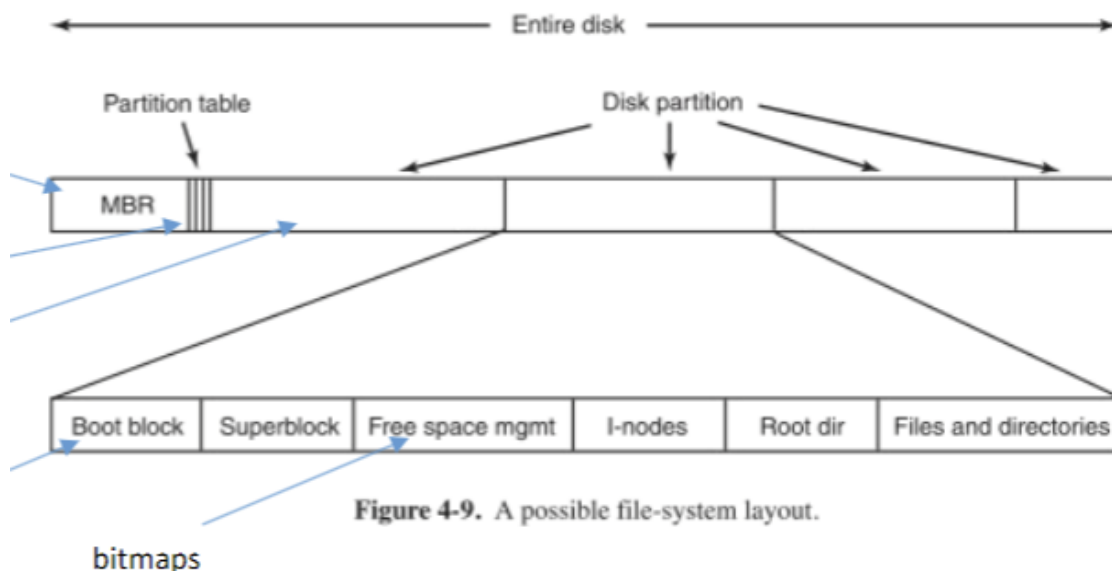
Un volumen coincide con un dispositivo físico. Dentro de él, se pueden definir unidades lógicas(C: y D: en Windows). El sistema de ficheros es el formateo de la partición en cuestión

MBR:

- En todo volumen, el sector 0 es el Master Boot Record(MBR).
- El MBR tiene 512 bytes y no forma parte del sistema de ficheros del SO.
- Al final del MBR(ultimos 64 bytes) encontramos la tabla de particiones.
 - La tabla de particiones es única por dispositivo pero en un volumen puede haber varias particiones
- **Limitaciones:** Solo puede manejar discos de hasta 2TB y 4 particiones primarias.
- Para superar esto se desarrolla GUID partition table (GPT)

El sistema de ficheros se instancia en cada partición.

En todas ellas se reserva al principio el bloque de arranque por si se instala un SO y es la unidad sobre la que debe cargarse.



Superbloque: Contiene información sobre la organización de la partición

- Tamaño de Agrupación
- Tamaño de Superbloque
- Bitmaps

- Zona de i-nodos
- Número de i-nodos
- Número del primer i-nodo

En una máquina pueden convivir volúmenes con distintos sistemas de ficheros. Para simplificar la gestión, se añade una nivel superior, un sistema virtual de ficheros que unifica el acceso

Bitmaps

Hay 2 bitmaps, uno para i-nodos y otro para agrupaciones.

Bitmap de Agrupaciones:

- Contiene un bit por cada bloque para saber si está ocupado o no
- Numero bloques bitmap = $\frac{Tam\ Disco}{Tam\ Bloque} = \text{Número de Bits para el bitmap de agrupaciones}$
- Número bits = Número bytes x 8
- Numero bloques partición para bitmap = $\frac{Numero\ Bits\ Bitmap}{Tam\ Bloque\ en\ Bits}$

Ejemplo: Disco de 1TB, Tam Bloque de 4kB

- Num Bloques Bitmap = $\frac{2^{40}(1TB)}{2^{12}(4kB)} = 2^{28} \text{ bloques} \rightarrow \text{Se reservan } 2^{28} \text{ bits para el bitmap de agrupaciones}$
- Tam Bloque en Bits = $2^{12} \times 8 = 2^{12} \times 2^3 = 2^{15}$

Los i-nodos ocupan un tamaño fijo. Sin embargo los directorios no se sabe de antemano cuánto van a ocupar pero no es necesario.

Son fichero cualesquiera y se ubican en la última zona de memoria

Comando tune2fs -l: Podemos ver una gran cantidad de información del sistema de ficheros

i-nodo

Es la estructura más importante para la gestión de ficheros.

Contiene la lista de los bloques ocupados por un fichero.

No contiene el nombre del fichero.

Se sabe de que i-nodo se trata en función de su posición en el sistema de ficheros.

En un sistema de doce entradas con bloques de 4kB, solo podríamos tener ficheros de menos de 48kB.

Para poder usar mas, se podría aumentar el tamaño del i-nodo pero crea problemas de compatibilidad

Se utiliza la indirección. Se añade un nuevo campo al i-nodo, un puntero, que contiene el numero de bloque en el que se guardan los punteros originales.

Acabamos de aumentar la capacidad de la lista. 12 + los que quepan en el bloque.

Como en ext2 los numeros de bloque son de 4 bytes y los bloques ocupan 4kB.

Caben $2^{12} / 2^2 = 1024$ numeros mas.

Este primer puntero es el PIS

Si añadimos otro campo igual, conseguimos 1024^2 numeros mas. Esto se llama PID

Si añadimos otro campo igual, conseguimos 1024^3 numeros mas. Esto se llama PIT

Siguiendo el ejemplo anterior: Pasamos de 48 kB a $(12+1024+1024^2 + 1024^3) * 4\text{kB}$ que son casi 4TB.

En Linux:

- ext2
- ext3: Parecida a ext2 y añade funciones de journaling
- ext4: Puede manejar dispositivos de mas de 4TB
 - Numero de bloques de 48 bits
 - Se pueden asignar rangos consecutivos de bloques llamados extents

FAT

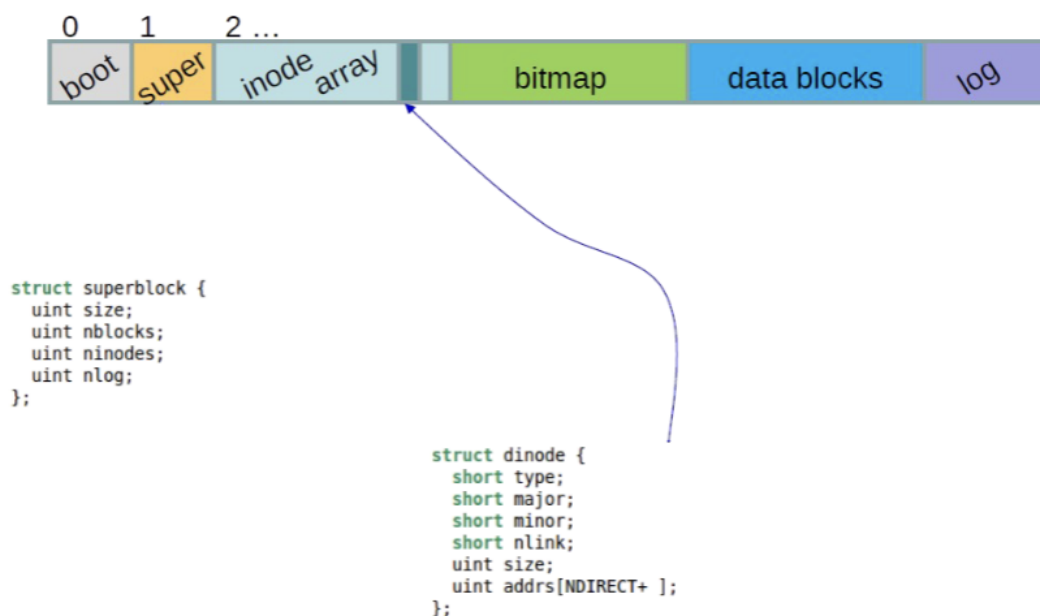
File Allocation Table: Sistema de registro de bloques en MS-DOS. Se sigue utilizando en USBs

FAT usa una lista enlazada de agrupaciones para llevar la contabilidad de bloques libres y ocupados.

Si el bloque esta libre se marca con el valor 0.

Si el bloque esta ocupado se marca con -1

Sistema de Ficheros en xv6



En xv6 se usa un tamaño de bloque igual al del sector de disco (512 bytes)

Servidor de Ficheros

Servidor de Ficheros: Parte del SSOO que ofrece al programador la abstracción FILE

Identificador: Número entero positivo que retorna la operación de apertura y con el proceso se relacionan con el servidor de ficheros

- Los identificadores abiertos por un proceso se guardan como vector en su BCP.

Cada entrada del vector mantiene un índice en una tabla única de ficheros abiertos.

Esta tabla apunta al i-nodo del fichero

Todos los procesos reservan los identificadores 0,1,2 para stdin, stdout y stderr respectivamente.

Apertura de Fichero

El SSOO recorre todo el directorio hasta localizar el nombre, sino lo encuentra devuelve NULL.

Después comprueba si el usuario tiene permisos de acceso con la información contenida en el i-nodo

Si es así, el i-nodo se instancia en la tabla de i-nodos kernel, se añade una entrada en la tabla global de ficheros y se escribe el número de esta entrada en el BCP del proceso que invocó fopen()

Creación de Fichero

Se comprueban los permisos de usuario en el directorio consultando la información en el directorio del i-nodo.

Si se dispone de ello, se busca un i-nodo libre en el bitmap de i-nodos, se instancia en memoria y se devuelve el valor del identificativo en la tabla de memoria

Lectura de Fichero

Requiere:

- Un descriptor de fichero abierto
- El número de bytes (N) que se van a leer y la dirección en la que se almacenaran

El puntero se aumenta en N

Escritura de Fichero

Permite escribir M bytes, devuelve el número de bytes realmente escrito

El servicio tiene que buscar agrupaciones libres para poder ampliar el fichero

Cierre de Fichero

Cuando se pide cerrar un fichero el SO invoca el cierre y la entrada de la tabla se marca como libre. Si no hay otros procesos usandolos, se libera también la información de la tabla de i-nodos

Montaje

Montaje: Mecanismo de Unix para poder ver todos los ficheros en un solo árbol

En los sistemas Unix, la jerarquía de ficheros es única, todos los directorios descienden en último término del raíz.

Caché de Ficheros

Caché de Ficheros: Zona de la RAM que si gestiona el SO de forma directa y utiliza la misma propiedad de proximidad referencial.

La operación más costosa es la escritura de discos.

Políticas de Sincronización:

- **Write Through:** Un bloque se escribe en cuanto se modifica. Equivale a que no haya caché
- **Write Back:** Los datos solo se escriben desde la caché cuando esta se haya llenado y hay que reemplazarlos
 - Si hay varios cambios en un mismo bloque, se podrían perderse si se pierde la alimentación
- **Delayed Write:** La sincronización se produce de forma periódica
- **Write on Close:** Se produce la sincronización cuando se cierra el fichero

Journaling

Cada vez que se llama a un servicio de disco, los cambios en sus metadatos se registran en el journal. Las entradas correspondientes a una mismo servicio constituyen una transacción. Cuando una operación individual se completa, por ejemplo escribir un bloque, se anota el resultado.

Si todas las operaciones de la transacción se cursan con éxito, los datos de esa transacción se borran del journal.

Si el sistema sufre un crash, el sistema puede saber en qué punto se suspendieron las transacciones que estaban abiertas y puede revertirlas. Esto ocurre en el arranque del sistema y es lo que explica por qué cuando apagamos nuestro equipo físicamente, el sistema

operativo tarda tanto en cargar en el siguiente arranque.

Tema 6 - Entrada/Salida

Introducción

El Sistema operativo oculta los detalles más complejos de los dispositivos electrónicos mediante la abstracción dispositivo que es una ampliación de la abstracción fichero.

La capa más próxima a los dispositivos son los drivers

Modelo de Entrada/Salida

Todos los dispositivos se conectan con la CPU usando protocolos

Bus: Conexión compartida por varios dispositivos que consiste en el envío de mensajes y señales eléctricas

La CPU se conecta con distintos buses usando distintos tipos de arbitraje llamado bridges.

Controlador: Microordenador empotrado en un dispositivo para conectarse con su mundo exterior.

La relación con otros dispositivos se hace usando un pequeño conjunto de dispositivos

¿Cómo sería la escritura de un fichero ya abierto?

- El driver recibe los datos de las capas superiores y escribe el siguiente carácter en el registro de datos del controlador.
- Este registro está fuera de CPU, por ello se hace un mapeo de direcciones
 - El Driver puedes leer y escribir ese registro
- Se pone en el registro de comandos la orden de escritura
- Pasado un rato aparecerá en el registro status el resultado de la operación

El controlador oculta toda la complejidad interna del dispositivo.

Direccionamiento

Port-Mapped IO:

- Algunos procesadores utilizan direccionamiento e instrucciones específicas para los dispositivos E/S.
- Esta solución tiene sentido para máquinas antiguas ya que su mapa de memoria era muy reducido y las direcciones un recurso escaso.

Memory-Mapped IO:

- No hay que usar instrucciones de código máquina distintas si accede a la RAM o a el registro del controlador
- x86 tiene el modo de puertos mapeados pero también permite este modo
- Hay 2 barreras para que este modo funcione:
 - Caché: Los dispositivos están direccionados en direcciones físicas pero la caché se interpone.
 - Para evitar esto, en la tabla de páginas del proceso aparecen la paginas con el caching disabled activado

Modos de Programación E/S

La adaptación entre las velocidades entre la CPU y el dispositivo es un problema serio.

La solución mas sencilla y ineficaz es la espera activa

Espera Activa: El driver lee en bucle infinito el registro de status has que cambia su estado

La E/S bloqueante tiene como inconveniente el cambio de contexto.

Si se genera poco trafico no hay problema pero si trata de transferir grandes bloques de información hay problemas

DMA (Direct Memory Address): Evita que los datos de transferencias masivas pasen por la CPU sino que fluyan entre los 2 dispositivos de forma transparente.

Para que no haya colisiones con las tareas que ya esta realizando el CPU se usan 2 técnicas:

- **Transferencia a Rafagas:** Desde que comienza hasta que acaba la transferencia, el bus queda en poder de los dispositivos
- **Robo de Ciclo:** Cada n ciclos de bus, el controlador del dispositivo origen activa una señal de robo que solicita acceso a la CPU y se desconecta del bus mientras dure la operación

Cuando termina la transferencia, el controlador DMA interrumpe a la CPU.

EL DMA no maneja direcciones virtuales

Dispositivos de Entrada/Salida

En Unix la entidad device puede ser de 3 tipos:

- **Dispositivos Orientados a Bloque:** Transfieren información en bloques de información de tamaño fijo y permiten el acceso aleatorio
- **Dispositivos Orientados a Carácter:** Aquellos que envían o reciben secuencias de caracteres. Se manejan con colas donde se van almacenando los caracteres y solo se puede leer el primero de ellos.
- **Dispositivos Orientados a Red:** Nacidos para atender dispositivos de red. Manejan paquetes de información de tamaño fijo pero el acceso no es aleatorio.

El Terminal

Terminal: Dispositivo de interacción con el usuario por excelencia. Consta de 2 dispositivos lógicos, el teclado y el monitor.

En dispositivos antiguos, cuando pulsamos un tecla se producía una interrupción y la CPU recibe el código de la tecla correspondiente y la convierte a ASCII.

Como la llegada era secuencial, se llama **modo crudo**.

En los teclados actuales, el envío se hace al pulsar la tecla INTRO ya que dispone de un microcontrolador y un buffer.

Este modo se llama **modo elaborado**.

La aplicación terminal se encarga de que cuando pulsas una letra en el teclado, aparezca el carácter correspondiente. Pero esto se puede desactivar.

Almacenamiento Secundario

Almacenamiento Secundario: Almacenamiento que persiste aunque se interrumpa la alimentación

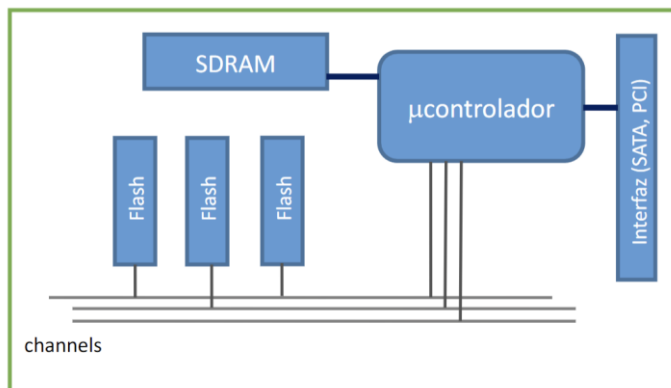
Nos centraremos en los discos de estado sólido ya que no tiene partes móviles

Usan tecnología de almacenamiento NAND flash.

La capacidad de almacenamiento por unidad de superficie con este sistema de ficheros es muy superior a la de la memoria SDRAM y no necesita refresco

Tiene tiempo de acceso mayor.

Componentes de un SSD:



- **Microcontrolador:** Pequeña CPU que contiene y ejecuta la inteligencia de la SSD
- **SDRAM:** Banco de memoria auxiliar del microcontrolador que actúa como caché
- **Interfaz:** Realiza diálogo con el driver
- **Banco de Memoria Flash:** Almacena los datos
- **Channels:** Buses internos de 8 bits que permite intercambio de información entre los bancos de memoria y el microcontrolador

FFS(Flash File System): Sistema de Ficheros propio del SSD que es transparente al programador de sistemas

FTL(Flash Translation Layer): Traduce bloques a posiciones físicas de la memoria RAM

Wear Leveling: Distribución de los ficheros por toda la memoria para no estresar unas pocas zonas

Los sectores que se quedan desocupados no se borran físicamente, simplemente se marcan como libres para poder ser reutilizados.

El microprocesador también realiza labores de compactación y borrado de bancos aislados con una rutina de garbage collector

Al igual que con los discos ópticos, los sectores se van degradando con el tiempo y el controlador debe marcarlos como defectuosos.

USB

USB(Universal Serial Bus): Interfaz más versátil para la comunicación de los dispositivos. En la versión 2.0, se usan 2 hilos para la transmisión de datos, con código de línea NRZ y dos hilos para la alimentación.

El modo de transmisión es half-duplex que se refiere a que se pueden enviar datos en ambos sentidos pero no simultáneamente.

La topología de USB es un árbol con un nodo raíz llamado **host** del que cuelgan otros llamados **slaves**.

Hub: Nodo esclavo que cuelga de otros.

La alimentación es asimétrica, sólo la proporciona el extremo que actúa como hub.

Cada dispositivo se direcciona con un end-point. El host es @1 ya que @0 está reservado.

Con USB se puede conectar dispositivos sin tener que apagar el PC.

USB ofrece 4 tipos de transferencia:

- **Control:** Se usa durante la configuración.
- **Interrupción:** Para dispositivos como ratón o teclado.
- **Masiva:** Para ficheros.
- **Isócrona:** Para audio/video.

La comunicación entre host y dispositivo se hace usando un protocolo con una trama especial llamada **token**.

Solo el propietario del token puede escribir en el bus.

USB es capaz de reconocer cualquier dispositivo al vuelo.

Protocolo de Sincronización:

1. El host detecta la conexión del nuevo dispositivo.
2. El host detecta si es de alta o baja velocidad.
3. El host resetea el dispositivo que toma el endpoint @0.
4. Se establece un canal entre el dispositivo y el host. El host pide que este que envíe su descriptor.
5. El dispositivo responde con su descriptor que contiene datos como tipo de equipo, fabricante, consumo etc.
6. Con esta información ya sabe que driver usar/instalar.
7. El host asigna un endpoint libre al dispositivo. Un mismo dispositivo puede tener varios endpoints.

Con USB 3.0 se mantiene la compatibilidad con 2.0 pero se modifican sustancialmente las posibilidades de estos.

El conector es de 8 hilos y permite comunicación full-duplex.

La alimentación es activa desde cualquiera de los extremos permitiendo así que se puedan cargar PC usando conectores tipo C.

Drivers en Linux

Driver: Capa del kernel que maneja los dispositivos

Sabemos cuántos dispositivos hay conectados gracias a un subdirectorio en /sys llamado module.

También se puede usar el comando lsmod para obtener información más detallada

Los dispositivos Linux se identifican por cifras mayor y menor.

Mayor: Identifica el tipo de dispositivo y sirve para saber que driver tiene que manejarlo

Minor: Distingue los edificios de la misma clase

Se puede ver esta información listando el directorio /dev

Cada dispositivo tiene asignado un inodo que almacena información sobre si es de tipo bloque o de tipo carácter y los números mayor y menor

Principios de Diseño de un Driver Linux:

- **API unificada:** Los dispositivos tienen que responder a los servicios open(), close(), read(), write() y seek(). El resto se hace con la función comodín ioctl().
- **Nombrado Coherente:** Los dispositivos generan entradas al directorio /dev y los nombres han de seguir un nombrado coherente
- **Buffering:** Los drivers actúan por memoria tampón para adecuar distintas velocidades de transferencias
- **Protección:** Los drivers tienen que soportar el mecanismo de control de acceso estándar para evitar que cualquier proceso de usuario pueda acceder a cualquier dispositivo físico

Los drivers de UNIX no entienden de formato de la información ya que solo ven flujos de bytes.

Los dispositivos se abren con el mismo servicio que los ficheros, con open()

La función devuelve el descriptor que se usa para todas las operaciones mientras que permanezca abierto o -1 si hay error.

El cierre se hace con close()

read() y write() realizan transferencia de información

Tema 7 - Comunicación y Sincronización

Introducción

El SO tiene que ofrecer mecanismos estandarizados, eficientes y seguros para la comunicación entre procesos.

La abreviatura IPC (Inter Process Communication) se usa de forma habitual para englobar todos los mecanismos

IPC

Señales

Señales: Mecanismo de comunicación asíncrona del kernel con los procesos.

Funcionan de forma parecida a una interrupción a nivel software. Si se está preparado para atenderla ejecuta la función en servicio de esa señal y en el retorno prosigue su flujo normal. Si no se dispone de función, se ejecuta la opción por defecto

Cada señal tiene un número asignado. Si tiene N/A, entonces el número depende del SO.

Hay señales como SIGUSR1 y SIGUSR2 que pueden ser usadas por el usuario con libertad.

Un proceso puede enviar una señal a otro siempre que pertenezcan al mismo usuario o grupo.

SIGINT: Cuando se hace control + c

SIGKILL: Resultado de la muerte de un proceso

SIGALRM: Ha vencido el temporizador

SIGCHLD: Proceso hijo muerto

Memoria Compartida

Servicio que proporciona el kernel a los procesos. Uno de ellos debe pedirlo de manera explícita.

El sistema reserva en RAM un rango que mapea como otro cualquiera en mapa de memoria virtual del peticionario

El acceso a la memoria requiere un clave que el peticionario puede compartir con otro proceso. Cuando esto ocurre, el SO también mapea esos marcos en las páginas de memoria virtual del segundo proceso.

Ocurrido esto, varios procesos pueden acceder tanto a la escritura como la lectura al buffer de memoria compartida.

La memoria compartida es un recurso crudo ya que no tiene ningún mecanismo de sincronización

Pipes

Pipes: Nacen para poder alimentar la salida de un proceso a la entrada del siguiente. Es secuencial, unidireccional y sin contención. No puede pararse al emisor desde el receptor salvo que se use una señal. Las tuberías son anónimas

Cuando hacemos fork(), el hijo hereda los descriptores abiertos del padre
Una manera de poner ambos en comunicación es crear 2 pipes desde el padre
El padre envía datos por uno y el hijo responde por el otro

Una extensión funcional de esta idea son las tuberías con nombre.
Se manejan de forma similar a los ficheros con path, nombre y comando de apertura, escritura, lectura y cierre.
Hay que borrarlas de forma explícita.
No pueden abrirse de forma lectura y escritura de manera simultánea. La lectura y la escritura son bloqueantes
Suelen ser arquitecturas tipo cliente/servidor

Colas de Mensajería

Colas de Mensajería: Tipo de comunicación asíncrona donde el envío y recepción no son bloqueantes
Utilizan un patrón publisher/subscriber
Se crean y se manejan como ficheros especiales que pueden tener nombre o ser anónimas.
Se instancia como una estructura de datos en el espacio del kernel. Este se encarga de sincronizar y evitar colisiones
Se tiene que crear expresamente. Pueden definirse como solo lectura, solo escritura o bidireccionales

Sockets

Sockets: Puntos lógicos de terminación de una canal de comunicación entre procesos
Hay 2 tipos de socket:

- **Datagram:** No orientados a sesión
- **Stream:** Orientados a conexión

Define el punto de acceso y los procedimientos para engancharse y gestionar el cambio de información

Stream Socket:

- No hay concepto de conexión
- El cliente hace una petición y el servidor responde.
- Garantiza reenvíos si hay fallos y garantiza los reenvíos en el mismo orden que se enviaron

Concurrencia

Concurrencia: Se produce cuando se pide el uso de un recurso limitado por parte de varios peticionarios.

Los recursos pueden ser:

- **Físicos:** Registro de un Controlador
- **Lógicos:** Campo de una base de datos
- **Reutilizables:** La operación no los destruye
- **Consumible:** El contenido de un mensaje
- **Exclusivo:** Un temporizador
- **Compartidos:** Un fichero
- **Expropiables:** Una página de memoria RAM
- **No expropiables:** Un mutex

Procesos Cooperantes: cuando están diseñados de forma premeditada para colaborar entre ellos

Procesos Independientes, cuando la concurrencia es fortuita

Cooperación Implícita: Cuando la cooperación la orquesta el sistema operativo de forma opaca al procesador

Cooperación Explícita: Cuando la cooperación sucede a nivel de procesos de usuario y se emplean los servicios de sincronización nativos del SO

Operación Atómica: Operación que para el resto del sistema ocurre de forma instantánea

- Son de código máquina ya que se ejecutan de principio a fin en la CPU
- La atomicidad se utiliza para crear mecanismos de sincronización

Sección Crítica: Un fragmento de código que se ejecuta de forma ininterrumpida desde el punto de vista de los procesos/threads concurrentes

Condición de Carrera: Se produce cuando varios procesos acceden a un mismo recurso y el orden de llegada es puramente aleatorio

Interbloqueo: Cuando un proceso retiene un recurso y no lo libera a la espera de que otro proceso que retiene pero no puede utilizar porque está a la espera del recurso anterior

Condiciones que tienen que ocurrir para que ocurra el interbloqueo:

- **Exclusión Mutua:** Hay un recurso por el que contienen los procesos y solo puede usarlo uno de ellos
- **Condición de Retención y Espera:** Los procesos mantienen la posesión de los recursos ya asignados mientras esperan recursos adicionales
- **Condición de No Expropiación:** Cuando un proceso ocupa el recurso no se le puede quitar.
- **Condición de Espera Circular**

Sincronización

Mutex

Mutex: Mecanismo bloqueante que funciona de forma automática utilizando instrucciones de ensamblador.

Garantiza que el thread adquiere la propiedad y ejecute la zona crítica sin interrupción por otros hilos del mismo sistema concurrente.

Cualquier otro hilo pasa a estado bloqueado hasta que el propietario libera el mutex

Mutex es muy eficaz pero también presenta problemas

Estampida: Si hay muchos threads bloqueados, la liberación del mutex lleva a todos a un estado Listo para ejecución, el scheduler elegirá a uno y después irán entrando en estado Bloqueado

- Esto lleva a muchos cambios de contexto improductivos
- La solución pasa por una cola en la que el kernel mantiene los threads bloqueados por un mismo mutex y solo despierta al que lleve más tiempo en espera

Inversión de Prioridad:

- Suponemos que hay 2 procesos que quieren acceder al mismo dato. Uno de prioridad alta y otro de prioridad baja
- Cuando el segundo se apropie del mutex, el primero va a estar bloqueado por un de baja prioridad.
- La solución es incrementar la prioridad de la tarea propietaria a la prioridad más alta

Spin-Lock

Spin-Lock: Mecanismo de bajo nivel que inhibe el kernel y parte las interrupciones y permite la ejecución atómica casi pura

Tiene mucho riesgo ya que si un thread que intenta capturar un spinlock quedará en espera activa si está ocupado

Recomendable para secciones críticas muy breves

Desaconsejable cuando la tarea lleva un tiempo impredecible

Semáforos

Semáforo: Sirve para arbitrar el acceso de M peticionarios a N recursos de forma simultánea tal que $M > N$

El semáforo implementa un contador de 0 a N. Cuando un proceso ocupa un recurso, el valor del contador se incrementa, cuando se desocupa, lo resta.

La operación de ocupar se denomina wait y la de liberar signal.

Tipo Test

Tema 4

1. **Tamaño de marco en xv6**
R: 4 KB
2. **Orden correcto de los elementos HW**
R: CPU - TLB - Caché - RAM
3. **¿Qué es el conjunto de trabajo?**
R: Fragmento de código y datos que se acceden de forma repetida durante un periodo breve
4. **Estructura de metadatos que gestiona la memoria de un proceso**
R: Tabla de memoria virtual
5. **¿Cómo se llama una zona de memoria inmune al swap?**
R: Pinned
6. **El kernel trabaja con direcciones virtuales una vez arrancado el sistema ¿Dónde se almacena esa correspondencia?**
R: En todas y cada una de las tablas de memoria de procesos
7. **¿Qué tipo de hardware se usa en la TLB?**
R: Memoria asociativa
8. **¿Para qué se usa la opción caching disabled?**
R: Para acceder a dispositivos de E/S mapeados en memoria
9. **Un microprocesador tiene una tabla de memoria virtual de 7 niveles ¿Cuántos niveles tiene la TLB?**
R: 1
10. **¿Cómo se llama la técnica para ahorrar espacio de memoria al hacer un fork()**
R: COW
11. **Bloque**
R: Datos de la RAM que se instancian en una línea de la caché
12. **¿Qué política de reemplazo no se usa en una caché con correspondencia directa?**
R: Ninguna de las otras opciones
13. **En una tabla de memoria de 5 niveles, ¿dónde se encuentra el número de marco?**
R: En la hoja final de cada rama
14. **¿Qué tipo de estructura para contabilizar los espacios libres y ocupados cuando se funciona con memoria real?**
R: Lista doblemente encadenada
15. **¿Cuándo funciona la TLB en modo real?**
R: Durante el arranque del sistema
16. **Elemento hardware que traduce la memoria virtual a memoria física**
R: TLB
17. **Registro de la CPU que apunta al inicio de la tabla de páginas del proceso en ejecución**
R: PTBR

18. **¿Cuántas instancias de una misma librería con linkado estático se cargan durante la ejecución de M procesos iguales?**

R: M

19. **¿Cuándo se recarga por completo la TLB?**

R: Cuando hay un cambio de contexto

20. **Inconveniente del algoritmo Worst Fit**

R: Obliga a recorrer toda la lista de espacios de memoria

Tema 5

1. **Conjunto de sectores contiguos en un disco magnético**

R: Pista

2. **Operación para crear un sistema de ficheros en una partición.**

R: Formateo

3. **¿Dónde se instala el cargador del Sistema Operativo?**

R: En el bloque 0 de la partición de arranque

4. **La función rewind()**

R: Posiciona el puntero de un fichero en el registro 0

5. **Identificador de fichero de stderr**

R: 2

6. **El puntero indirecto simple**

R: Contiene un número de agrupación

7. **¿Cuántos i-nodos ocupa un fichero de 4MB si el tamaño del bloque son 4kB?**

R: 1

8. **Tamaño de los i-nodos**

R: Ninguna de las otras respuestas

9. **En un directorio la entrada especial llamada .. Apunta**

R: Al directorio padre

10. **Una agrupación es..**

R: Un conjunto de sectores

11. **La tabla de particiones...**

R: Se encuentra en el MBR

12. **¿Qué es un punto de montaje?**

R: El directorio del anfitrión bajo el que aparece el sistema de ficheros montado

13. **¿Cuándo se revierten las transacciones erróneas en un sistema con journaling?**

R: Durante el siguiente arranque

14. **La tabla intermedia de descriptores de ficheros**

R: Todas las demás respuestas son válidas

15. **Si se borra un enlace simbólico en Linux**

R: El fichero apuntado no se altera

16. **¿Dónde se encuentra la información de agrupaciones ocupada por un fichero?**

R: En el i-nodo

17. **En un directorio la entrada especial llamada . Apunta**

R: Al propio directorio

18. **¿Qué estructura de datos usa FAT para contabilizar los bloques ocupados por un fichero?**
R: Una lista
19. **Tamaño de bloque en xv6**
R: 512 bytes
20. **¿Qué equivalencia tiene una partición Unix en Windows?**
R: Unidad lógica

Tema 6

1. **¿Qué dirección tiene un dispositivo USB durante el handshake?**
R: 0
2. **Circuito que conecta la CPU con un bus**
R: Bridge
3. **¿Cómo se llama el nodo raíz de un bus USB?**
R: Host
4. **¿Cuántos hilos tiene un cable USB 3?**
R: 8
5. **Si estás desarrollando un driver, ¿qué función no puedes invocar?**
R: Ninguna de las anteriores
6. **¿Cómo se llama la técnica DMA para compartir el bus con la CPU?**
R: Robo de ciclo
7. **¿Qué servicio debe ofrecer obligadamente un driver de Linux?**
R: Open
8. **¿Cómo se llama la técnica de los SSDs para prolongar la vida de los bancos de memoria?**
R: Wear leveling
9. **Interfaz de un dispositivo físico con el mundo exterior**
R: Controlador
10. **Comando Linux para ver los drivers cargados**
R: lsmod

General

1. **Conjunto de sectores magnéticos contiguos en un SSD**
R: Ninguna de las anteriores es cierta
2. **El puntero indirecto triple**
R: Contiene un número de bloque
3. **Identificador del fichero stdin**
R: 0
4. **Inconveniente del algoritmo First Fit en la gestión de memoria real**
R: Tiende a producir más fragmentación que otras políticas de asignación
5. **La función fseek()**
R: Posiciona el puntero de un fichero en la posición deseada
6. **La tabla de particiones...**
R: Es única por volumen

- 7. La tabla intermedia de descriptores de ficheros**
R: Es única por sistema
- 8. Operación para crear un sistema de ficheros**
R: Formateo
- 9. Si se crea un enlace simbólico en Linux**
R: El fichero apuntado no se altera
- 10. Si una partición en un sistema tipo Unix tiene capacidad para 2^{27} ficheros, ¿cuántos Bytes ocupará el bitmap de inodos?**
R: 2^{24}
- 11. Tamaño de los inodos en NTFS**
R: Ninguna de las otras respuestas
- 12. Tamaño por defecto de la página de memoria virtual en Linux**
R: 4 kB
- 13. Tipo de estructura de datos que se usa para registrar los espacios libres y ocupados en una memoria física**
R: Lista doblemente encadenada
- 14. Una agrupación en UNIX es ...**
R: Lo mismo que un bloque
- 15. ¿Cuándo se revisan las transacciones no completadas en un sistema con journaling?**
R: Durante el siguiente arranque
- 16. ¿Cuántos superbloques hay por partición?**
R: 1
- 17. ¿Cómo se llama una zona de memoria que el programador "marca" para que nunca pueda expulsarse a la zona de swap del disco?**
R: Pinned
- 18. ¿De las siguientes señales cuál no es enmascarable?**
R: SIGKILL(9)
- 19. ¿Dónde encontrarías el número del inodo correspondiente a un nombre de fichero concreto?**
R: En el directorio
- 20. ¿Dónde se cargan en el mapa de memoria virtual de un proceso las variables definidas antes del main() de un programa C?**
R: En una contigua al texto
- 21. ¿En una tabla de memoria virtual de 2 niveles, dónde se encuentra el número de marco?**
R: En la hojas del nivel 2 del árbol de memoria